Activation Service

Guia de QA: estratégia, casos de teste manuais e rastreabilidade com a automação

Este guia orienta a validação manual do serviço. Ele não repete o que a automação já garante: explica **o que testar manualmente e por quê**, prioriza pelo risco ao negócio e liga cada caso ao teste automatizado correspondente, para que o tempo de QA se concentre onde a automação não alcança.

67 casos em 8 áreas	**22 P1** 27 P2 · 18 P3	**84%** com contraparte automatizada	**22/22** P1 também automatizados

5
sessões exploratórias

Sumário

1. Objetivo e escopo
2. Estratégia de teste
3. Priorização por risco
4. Ambiente e dados de teste
5. Critérios de entrada e de saída
6. Casos de teste por área
7. Testes exploratórios
8. Comportamentos conhecidos (não são defeitos)
9. Registro de defeitos
10. Matriz de rastreabilidade

1. Objetivo e escopo

Objetivo: confirmar, com o sistema montado de ponta a ponta, que os requisitos do serviço se comportam como especificado do ponto de vista de quem o consome e de quem o opera.

Dentro do escopo	Fora do escopo
• Os 3 endpoints da API e o contrato HTTP • Autenticação, autorização e isolamento entre sistemas • Duplicidade e idempotência, inclusive concorrente • Processamento assíncrono e cada tipo de resposta do provedor • Recuperação de falhas de infraestrutura • Formato de erros, limites e observabilidade	• Testes de carga e desempenho sustentado • Testes de penetração completos • O provedor real e o servidor OAuth real (simulados no ambiente) • Infraestrutura de produção (cluster, gateway, rede)

2. Estratégia de teste

O projeto tem 188 testes automatizados que rodam a cada build. O papel do QA manual é **complementar**, não duplicar:

Camada	O que cobre	Papel do QA manual
Automação unitária e de integração	Regras de negócio, contrato HTTP, banco real, provedor simulado, segurança, arquitetura	Confirmar que a automação está verde antes de começar (critério de entrada)
Testes manuais roteirizados (seção 6)	O sistema montado de verdade: containers, rede, login OAuth real, Swagger/Postman	Validar a experiência de quem consome e de quem opera; cobrir os casos marcados como **manual** e **parcial**
Testes exploratórios (seção 7)	O que nenhum roteiro prevê	Buscar defeitos em entradas inesperadas, timing e falhas combinadas

Cada caso traz a coluna **Automação**: `auto` há teste automatizado equivalente (o caso manual confirma o comportamento no ambiente integrado); `parcial` a automação cobre parte do cenário; `manual` só o teste manual cobre — esses merecem mais atenção.

3. Priorização por risco

A prioridade de cada caso vem do impacto de uma falha naquele comportamento:

Prioridade	Critério	Exemplos
P1	Falha causa dado errado, perdido ou duplicado, ativação indevida ou acesso a dados de outro sistema	Duplicidade concorrente, isolamento entre clientes, tratamento de cada resposta do provedor, recuperação após queda
P2	Falha confunde o consumidor ou o operador, sem corromper dados	Código HTTP errado, validação incompleta, rastreio por `traceId`, probes
P3	Falha cosmética ou em caso raro	Página além do fim, campo desconhecido, documentação pública

Ordem de execução recomendada: todos os P1 primeiro; se o tempo acabar, a rodada ainda terá coberto o que mais importa.

4. Ambiente e dados de teste

1. Docker Desktop em execução; no terminal, na pasta do projeto: `docker compose down -v` e `docker compose up -d --build`.
2. Aguardar `docker compose ps` mostrar `app` como `(healthy)`.
3. Ferramenta principal: Swagger UI em `http://localhost:18080/swagger-ui.html` (ou a coleção Postman em `docs/postman`). Alguns casos usam o terminal; eles dizem isso. Os traces ficam no Jaeger, em `http://localhost:18686`.

Cliente (`client_id`)	Escopos	Uso
`orders-system`	read + write	Fluxo normal
`reporting`	read	Permissões
`billing-system`	read + write	Isolamento
`lista-qa`	read + write	Listagem

`productCode`	Provedor simulado
qualquer outro	sucesso (300 ms)
`INVALID-PRODUCT`	recusa (400)
`UNAVAILABLE-PRODUCT`	503 + `Retry-After: 5`
`SLOW-PRODUCT`	8 s (acima do limite de 5 s)

O servidor OAuth de teste aceita qualquer `client_secret`; as permissões vêm dos escopos marcados. "Corpo padrão" = `{"externalId": "<novo>", "customerId": "CUST-1", "productCode": "INTERNET-500"}`, com um `externalId` novo por caso (`QA-CRI-01`, ...). Para os casos de terminal, obtenha um token com:

```
TOKEN=$(curl -s -u orders-system:x -X POST localhost:18090/default/token -d
grant_type=client_credentials -d "scope=activation:read activation:write" | grep -o
'"access_token" *: *"[^"]*"' | cut -d'"' -f4)
```

5. Critérios de entrada e de saída

Entrada (para começar)

- `./mvnw verify` verde: testes, arquitetura e gate de cobertura
- Stack no ar e `healthy`
- Banco limpo (`down -v`)
- Versão (commit) anotada no registro da rodada

Saída (para aprovar)

- 100% dos **P1** passaram
- Nenhum defeito de severidade Alta aberto
- ≥ 95% dos **P2** passaram; cada falha com defeito registrado
- As 5 sessões exploratórias executadas, com notas

6. Casos de teste por área

Marque a coluna **OK** ao executar. Todo erro deve vir como `application/problem+json` com `correlationId` e `traceId`, mesmo quando o caso não repete isso.

6.1 Autenticação e autorização R8

Quem é você (401), o que pode fazer (403) e quais dados enxerga.

ID	Prio.	Cenário e passos	Resultado esperado	Automação	OK
AUT-01	P1	**Acesso sem login** Sem Authorize, executar `GET /api/v1/requests`	`401` ; header `www-authenticate` começa com `Bearer` ; corpo com `type /unauthorized`	auto `ActivationRequestControllerTest.requestWithoutTokenIsRejectedWith401`	☐
AUT-02	P1	**Login com permissão completa** Authorize como `orders-system` com os dois escopos; `GET /api/v1/requests`	"Authorized", cadeados fechados, `200`	auto `JwtValidationIT.validTokenIsAccepted`	☐
AUT-03	P1	**Só leitura tenta criar** Authorize como `reporting` só com `activation:read` ; `POST` com corpo padrão	`403` `/forbidden` ; nada é criado	auto `ActivationRequestControllerTest.readScopeCannotCreate`	☐
AUT-04	P2	**Só leitura consulta** Com o login de AUT-03, `GET /api/v1/requests`	`200`	auto `ActivationRequestControllerTest.existingRequestReturns200`	☐
AUT-05	P1	**Só escrita tenta consultar** Authorize só com `activation:write` ; `GET /api/v1/requests`	`403`	auto `ActivationRequestControllerTest.writeScopeCannotRead`	☐
AUT-06	P2	**Token sem escopo** Authorize sem marcar escopos; `GET` e `POST`	`403` nos dois	manual	☐

AUT-07	P1	**Token inválido ou forjado**
		`curl` com `Authorization: Bearer abc`
			`401` , sem explicar o motivo	auto
				`ActivationRequestControllerTest.invalidTokenIsRejectedWith401`
				`JwtValidationIT.forgedSignatureIsRejected`
					☐

AUT-08	P2	**Token expirado** Login, esperar 16 min, `GET`	`401` ; novo login volta a `200`	auto `JwtValidationIT.expiredTokenIsRejected`	☐
AUT-09	P3	**Documentação pública** Aba anônima: `/v3/api-docs`	`200` sem login	auto `OpenApiIT.apiDescriptionIsPublicAndAnnouncesTheClientCredentialsFlow`	☐

6.2 Criação e validação R1

Campos obrigatórios, 1 a 64 caracteres, só letras, números, `.` , `_` e `-` .

ID	Prio.	Cenário e passos	Resultado esperado	Automação	OK

CRI-01	P1	**Criação válida**
		`POST` com corpo padrão
			`201` ; `location` com o `id` ; `PENDING` ; datas UTC terminando em `Z`	auto
				`ActivationRequestControllerTest.validRequestReturns201WithLocation`
				`ActivationRequestApiIT.createdRequestIsPersistedAndCanBeRetrieved`
					☐

CRI-02	P1	**Campo obrigatório ausente** `POST` sem `customerId`	`400` ; `errors` com `customerId` / `is required`	auto `ActivationRequestControllerTest.missingAndInvalidFieldsReturn400WithFieldErrors`	☐
CRI-03	P2	**Campo só com espaços** `"productCode": " "`	`400` no campo `productCode`	auto `ActivationRequestControllerTest.blankFieldReturns400`	☐
CRI-04	P2	**Caracteres proibidos** `externalId` = `EXT 1` e depois `EXT#1`	`400` nos dois, mensagem de formato	auto `ActivationRequestControllerTest.missingAndInvalidFieldsReturn400WithFieldErrors`	☐
CRI-05	P2	**Limite de tamanho** `externalId` com 64 e com 65 caracteres	64: `201` ; 65: `400` `at most 64`	parcial `ActivationRequestControllerTest.missingAndInvalidFieldsReturn400WithFieldErrors(65)`	☐
CRI-06	P3	**Caracteres especiais permitidos** `externalId` = `qa.CRI_06-x`	`201`	manual	☐
CRI-07	P3	**Corpo vazio** `POST` com `{}`	`400` com os três campos	manual	☐
CRI-08	P2	**JSON malformado** Corpo padrão sem a `}` final	`400` `/malformed-request` , sem detalhes do parser	auto `ActivationRequestControllerTest.malformedJsonReturns400WithoutParserDetails`	☐
CRI-09	P3	**Campo desconhecido** Corpo padrão + `"extra": "x"`	`201` ; resposta sem `extra`	manual	☐

CRI-10	P3	**Ordem estável dos erros**
		`{"externalId": "EXT 1", "productCode": ""}` , enviado 3 vezes
			`errors` sempre na mesma ordem: `customerId` , `externalId` , `productCode` (por campo e, no mesmo campo, por mensagem)	auto
				`ActivationRequestControllerTest.missingAndInvalidFieldsReturn400WithFieldErrors`
				`ActivationRequestControllerTest.blankFieldReturns400`
					☐

6.3 Duplicidade e idempotência R4

O mesmo `externalId` nunca gera dois registros. Base: criar `QA-DUP-01`.

ID	Prio.	Cenário e passos	Resultado esperado	Automação	OK
DUP-01	P1	**Reenvio idêntico (retry)** Repetir o `POST` de `QA-DUP-01`	`200`, mesmo `id`, sem `location`	auto `ActivationRequestApiIT.retryWithSameContentReturns200WithTheSameRequest`	☐
DUP-02	P1	**Mesmo id, outro produto** `QA-DUP-01` com `TV-100`	`409` `/duplicate-request`	auto `ActivationRequestApiIT.sameExternalIdWithDifferentContentReturns409`	☐
DUP-03	P2	**Mesmo id, outro cliente final** `QA-DUP-01` com `CUST-2`	`409`	auto `ActivationRequestServiceTest.sameExternalIdWithDifferentContentIsRejected`	☐
DUP-04	P1	**Id já usado por outro sistema** Login como `billing-system`; mesmo `POST`	`409` genérico, sem dados do registro original	auto `ActivationRequestApiIT.externalIdTakenByAnotherClientReturnsGenericConflict`	☐
DUP-05	P2	**Reenvio após concluído** Esperar `COMPLETED` e reenviar	`200`, mesmo `id`, `COMPLETED`; sem nova ativação	manual	☐
DUP-06	P3	**Maiúsculas e minúsculas** `qa-dup-06` e `QA-DUP-06`	Dois `201`, ids diferentes	manual	☐
DUP-07	P1	**Envios simultâneos** 10 `POST` idênticos em paralelo (terminal)	Um `201`, nove `200`, um único registro	auto `DuplicateRequestConcurrencyIT.concurrentIdenticalRequestsCreateExactlyOneRecord`	☐

6.4 Consulta e isolamento R2, R8.4

Cada sistema enxerga só o que criou; para os outros, o registro não existe.

ID	Prio.	Cenário e passos	Resultado esperado	Automação	OK
CON-01	P1	**Consulta existente** Criar e consultar pelo `id`	`200` com todos os campos	auto `ActivationRequestApiIT.createdRequestIsPersistedAndCanBeRetrieved`	☐
CON-02	P2	**Datas da criação e da consulta** Comparar `createdAt` do POST e do GET	Idênticos até os milissegundos	auto `ActivationRequestApiIT.timestampsReturnedOnCreationMatchTheStoredOnes`	☐
CON-03	P2	**Id inexistente** `GET` com UUID de zeros	`404` `/not-found`	auto `ActivationRequestApiIT.unknownIdReturns404`	☐
CON-04	P3	**Id em formato inválido** `GET` com `abc`	`404` (não 400 nem 500)	auto `ActivationRequestControllerTest.malformedIdReturns404`	☐
CON-05	P1	**Outro sistema tenta ler** Login como `billing-system`; `GET` do id de CON-01	`404`, igual a inexistente	auto `ActivationRequestApiIT.anotherClientsRequestIsNotFound`	☐
CON-06	P3	**Motivo de falha só em FAILED** Comparar um `COMPLETED` com um `FAILED`	Só o `FAILED` tem `failureReason`	parcial `ActivationProcessingIT.badRequestFailsWithoutRetry`	☐

6.5 Listagem, filtros e paginação R3

Base: cliente `lista-qa` com 5 pedidos (3 de `CUST-A`, 2 de `CUST-B`, um destes com `INVALID-PRODUCT`).

ID	Prio.	Cenário e passos	Resultado esperado	Automação	OK
LIS-01	P2	**Sem filtros** `GET /api/v1/requests`	Os 5, mais recente primeiro	auto `ActivationRequestListingIT.listsOnlyTheCallersRequestsNewestFirst`	☐
LIS-02	P2	**Por status** `status=FAILED`	Só o de `INVALID-PRODUCT`	parcial `ActivationRequestListingIT.filtersByStatusAndCustomerCombined`	☐
LIS-03	P2	**Por cliente final** `customerId=CUST-A`	Os 3 de `CUST-A`	auto `ActivationRequestListingIT.filtersByCustomer`	☐
LIS-04	P3	**Filtros combinados** `COMPLETED` + `CUST-B`	1 item	auto `ActivationRequestListingIT.filtersByStatusAndCustomerCombined`	☐
LIS-05	P2	**Paginação** `size=2`, páginas 0, 1 e 2	2, 2 e 1 itens, sem repetição; `totalPages` = 3	auto `ActivationRequestListingIT.paginates`	☐
LIS-06	P3	**Página além do fim** `page=99`	`200` com lista vazia	manual	☐

LIS-07	P3	**Tamanho fora do limite**
		`size=101`, `size=0`, `page=-1`
			`400` nos três	auto
				`ActivationRequestControllerTest.sizeAboveMaximumReturns400`
				`negativePageReturns400`
					☐

LIS-08	P3	**Status inexistente** `status=DONE` e `pending` (terminal)	`400` no campo `status`	auto `ActivationRequestControllerTest.invalidStatusReturns400`	☐
LIS-09	P1	**Isolamento na listagem** Login como `billing-system`; listar	Nenhum pedido de `lista-qa`	auto `ActivationRequestApiIT.listingIsScopedToTheCaller`	☐

6.6 Processamento e integração com o provedor R5

A criação responde `PENDING`; o worker chama o provedor em até 1 s. Acompanhe com `docker compose logs -f app`. PRO-03, PRO-04 e PRO-09 abrem o circuit breaker: rode-os por último ou espere ~30 s antes do próximo caso.

ID	Prio.	Cenário e passos	Resultado esperado	Automação	OK

PRO-01	P1	**Ativação com sucesso**
		`INTERNET-500`; consultar após ~2 s
			`COMPLETED`, `updatedAt` > `createdAt`	auto
				`ActivationProcessingIT.successCompletesTheRequest`
				`ScheduledProcessingIT.createdRequestIsActivatedInTheBackground`
					☐

PRO-02	P1	**Recusa definitiva** `INVALID-PRODUCT`	`FAILED`, "HTTP 400", uma única tentativa	auto `ActivationProcessingIT.badRequestFailsWithoutRetry`	☐
PRO-03	P1	**Indisponível com Retry-After** `UNAVAILABLE-PRODUCT`; consultar após 2 s e após ~90 s	`PENDING` com tentativas a cada ~5 s e uma pausa de 30 s do circuit breaker no meio; depois `FAILED` "Gave up after 5 attempts"	auto `ActivationProcessingIT.throttlingHonoursRetryAfterGivesUpAfterMaxAttempts`	☐
PRO-04	P1	**Timeout** `SLOW-PRODUCT`; consultar após ~2 min	"Provider call timed out" em cada tentativa; `FAILED` no fim	auto `ActivationProcessingIT.timeoutIsRetriedWithTheSameIdempotencyKey`	☐
PRO-05	P2	**A API não espera o provedor** `POST` com `SLOW-PRODUCT` medindo o tempo (`curl -w`)	Bem abaixo de 1 s, apesar dos 8 s do provedor	manual	☐
PRO-06	P2	**Headers enviados ao provedor** Criar com `X-Correlation-Id`; abrir `:18089/__admin/requests`	`Idempotency-Key` = `id`; mesma correlação	auto `ProviderClientTest.sendsIdempotencyKeyCorrelationIdCredentialAndBody`	☐
PRO-07	P2	**Mesma chave em todas as tentativas** Ver as chamadas do PRO-03 no WireMock	Mesma `Idempotency-Key` em todas	auto `ActivationProcessingIT.serverErrorIsRetriedUntilSuccess`	☐

PRO-08	P1	**Provedor recusa a nossa credencial (401)**
		Criar um stub temporário: `POST :18089/__admin/mappings` com `{"priority":1,"request":{"method":"POST","url":"/provider/v1/activate","bodyPatterns":[{"contains":"QA-401"}]},"response":{"status":401}}`; criar com `productCode` = `QA-401`; acompanhar por 3 min; apagar o stub (`DELETE :18089/__admin/mappings/{id}`)
			Continua `PENDING` (nunca `FAILED`), nova chamada a cada ~1 min; log `ERROR` com `Check PROVIDER_API_KEY`; `outcome="credentials_rejected"` em `/actuator/prometheus`; após apagar o stub, `COMPLETED` em até ~1 min	auto
				`ActivationProcessingIT.rejectedCredentialsNeverFailTheRequest`
				`ProviderResponsesTest.authenticationErrorsAreNotBlamedOnTheRequest`
					☐

PRO-09	P1	**Circuit breaker**
		Criar 6 com `UNAVAILABLE-PRODUCT` e logo 1 com `INTERNET-500`; após ~5 s ver o log e o banco (`psql`: `status`, `attempts`)
			Log `State transition from CLOSED to OPEN`; as solicitações que vieram depois ficam `PENDING` com `attempts = 0` (nenhuma tentativa gasta); ~30 s depois, uma chamada de teste (`HALF_OPEN`)	auto
				`ActivationProcessingIT.providerOutageOpensTheCircuit`
				`ActivationWorkerTest.circuitOpeningMidBatchReleasesTheRestWithoutCalling`
					☐

6.7 Resiliência e operação R5.7, R9

Partes do ambiente são derrubadas de propósito para ver o serviço se recuperar sozinho.

ID	Prio.	Cenário e passos	Resultado esperado	Automação	OK
OPE-01	P2	**Saúde do serviço** `:18081/actuator/health`, `/liveness`, `/readiness`	Os três `UP`	manual	☐
OPE-02	P3	**Actuator fora da porta pública** `curl localhost:18080/actuator/health`	`401` sem token, `404` com token	manual	☐
OPE-03	P2	**Banco fora do ar** `docker compose stop postgres`; probes e um `POST`; depois `start`	Readiness `503`, liveness `200`, `POST` `503` com `Retry-After: 5` em ~3 s; volta sozinho	parcial `ApiExceptionHandlerTest.databaseOutageIsA503WithRetryAfter`	☐
OPE-04	P1	**Provedor fora do ar** `stop provider-mock`; criar; `start` em até 10 s	"Provider is unreachable" com retry; depois `COMPLETED`	parcial `ProviderClientTest.unreachableProviderIsTransient`	☐
OPE-05	P1	**Reinício durante o processamento** `SLOW-PRODUCT` e logo `docker compose restart app`	Nada se perde; nunca fica `PROCESSING` mais de ~2 min	parcial `ActivationProcessingIT.expiredClaimIsReleased`	☐
OPE-06	P3	**Métricas** `:18081/actuator/prometheus`, filtrar `activation_`	Chamadas por `outcome`, transições por status e fila atual (`activation_requests_active` com `PENDING` e `PROCESSING`)	parcial `ActivationWorkerTest.providerCallsAreTimedByOutcome` `ActivationProcessingIT.queueGaugeFollowsTheTable`	☐
OPE-07	P2	**Rastreio de ponta a ponta** Criar com `X-Correlation-Id: qa-ope-07`; `grep qa-ope-07` nos logs; abrir o `traceId` do log no Jaeger (`:18686`)	Log da criação e do worker com o mesmo `correlationId` e `traceId`; no Jaeger, um único trace com o `POST`, o `activation process` e a chamada ao provedor	auto `ActivationRequestApiIT.creationIsLoggedWithTheCorrelationId` `TracingIT.providerCallBelongsToTheCreatingTrace`	☐
OPE-08	P2	**Servidor de autenticação fora do ar** `docker compose restart app`; obter um token; `docker compose stop auth`; só então o primeiro `GET` com o token (terminal); depois `docker compose start auth`. As chaves ficam em cache após a primeira validação, por isso a ordem	`503` com `Retry-After: 5` e `/service-unavailable` (não `401`); log dizendo que as chaves do emissor estão inacessíveis; com o auth de volta, `200`	auto `JwksOutageIT.unreachableKeysAreAnOutageNotAnInvalidToken` `SecurityProblemHandlerTest.unreachableIssuerKeysAreA503WithRetryAfter`	☐

6.8 Formato de erros e limites R7, R8.5

Todo erro sai em `application/problem+json` com `correlationId` e `traceId`.

ID	Prio.	Cenário e passos	Resultado esperado	Automação	OK

ERR-01	P2	**Formato padrão**
		Qualquer `400` (ex.: CRI-02)
			`problem+json` com `type`, `title`, `status`, `detail`, `instance`, `correlationId` e `traceId`	auto
				`ActivationRequestControllerTest.missingAndInvalidFieldsReturn400WithFieldErrors`
				`TracingIT.errorResponseCarriesTheTraceId`
					☐

ERR-02	P3	**correlationId igual ao header** Comparar `correlationId` do corpo e `x-correlation-id`	Iguais	auto `ActivationRequestControllerTest.suppliedCorrelationIdIsEchoedAndReturnedInErrors`	☐
ERR-03	P3	**Correlação do cliente** `POST {}` com `X-Correlation-Id: qa-err-03`	`correlationId` = `qa-err-03`	auto `ActivationRequestControllerTest.suppliedCorrelationIdIsEchoedAndReturnedInErrors`	☐
ERR-04	P2	**Correlação maliciosa** `X-Correlation-Id: <script>`	Descartada; id novo de 32 hexadecimais	auto `ActivationRequestControllerTest.correlationIdOutsideTheAllowedFormatIsReplaced`	☐
ERR-05	P3	**Método não suportado** `DELETE` num id	`405` em `problem+json` (`type` `about:blank`)	auto `ActivationRequestControllerTest.unsupportedMethodAlsoUsesProblemDetailWithTraceId`	☐
ERR-06	P3	**Tipo de conteúdo errado** `Content-Type: text/plain`	`415` em `problem+json`	manual	☐
ERR-07	P2	**Corpo acima de 16 KB** Corpo padrão + campo de 17 000 caracteres	`413` `/payload-too-large`; nada criado	auto `ActivationRequestControllerTest.oversizedBodyIsRejectedWith413`	☐
ERR-08	P2	**Header com caractere de controle** `X-Correlation-Id: a\x01b` (terminal)	`400`, nunca `500`	auto `ActivationRequestControllerTest.headerWithControlCharactersIsRejectedAs400`	☐
ERR-09	P2	**Limite de chamadas** 80 `GET` em paralelo com o mesmo token (terminal: `curl -Z`), depois mais um; repetir com outro `client_id`	Parte responde `429` com `Retry-After: 1` e `/too-many-requests`; o outro cliente não é afetado; 1 s depois volta a `200`	auto `RateLimitTest.clientOverItsLimitIsThrottledWithoutAffectingOthers`	☐

7. Testes exploratórios

Sessões de 20 a 30 minutos guiadas por uma missão, não por passos. Anote o que foi tentado, o que foi observado e as dúvidas — o valor está no que o roteiro não previu.

EXP-01 · Entradas fora do padrão

Missão: explorar a criação com entradas que um sistema real poderia enviar: acentos e emoji, espaço no fim, `null` explícito, número no lugar de texto, campos duplicados no JSON, corpo muito aninhado.
Hipótese: toda entrada vira `201` coerente ou `400` com `errors` claros — nunca `500`.

EXP-02 · Concorrência com conteúdos diferentes

Missão: disparar em paralelo (Postman Runner ou terminal) envios do mesmo `externalId` com `productCode` alternados.
Hipótese: exatamente um registro; um `201`, os demais `200` ou `409` conforme o conteúdo — nunca dois registros nem `500`.

EXP-03 · Tokens adulterados

Missão: modificar o payload de um token válido sem reassiná-lo (trocar `client_id`, `scope`, `exp`, `aud`), trocar o `alg`, remover a assinatura.
Hipótese: qualquer adulteração resulta em `401`, sem pistas sobre o motivo.

EXP-04 · Ciclo de vida sob perturbação

Missão: combinar quedas em momentos diferentes — reiniciar o app, parar o provedor, parar o banco — com pedidos em todos os estados.
Hipótese: nenhum pedido fica parado para sempre. Verifique ao final: `SELECT status, count(*) FROM activation_request GROUP BY status;` e procure `PROCESSING` com mais de 2 minutos (ou acompanhe `activation_requests_active` em `/actuator/prometheus`).

EXP-05 · Observabilidade do ponto de vista do suporte

Missão: para cada falha provocada nas sessões anteriores, partir só do `traceId` devolvido ao cliente e reconstruir o que aconteceu pelos logs.
Hipótese: o `traceId` sempre leva à causa, nos logs e no Jaeger; as mensagens dizem o que aconteceu e nenhum token ou credencial aparece nos logs.

8. Comportamentos conhecidos (não são defeitos)

Decisões de projeto documentadas no README e nos ADRs. Se algum deles parecer errado do ponto de vista do negócio, registre como **sugestão**, não como defeito.

Comportamento	Motivo
`externalId` diferencia maiúsculas de minúsculas	Identificadores do sistema de origem são tratados como opacos
Unicidade do `externalId` é global, entre todos os sistemas	Leitura literal do enunciado; evolução registrada (ADR-0002)
Retry idêntico responde `200`, não `409`	Idempotência para quem faz retry (ADR-0002)
Dado de outro sistema responde `404`, não `403`	Não revelar a existência do registro (OWASP API1)
Com o banco fora, ou com o servidor de autenticação fora quando as chaves não estão em cache, a API responde `503` com `Retry-After` (o banco em ~3 s)	Indisponibilidade temporária, não erro do cliente; o pool desiste rápido em vez de prender a requisição
`405` e `415` têm `type` = `about:blank`	Padrão do RFC 9457 para erros genéricos do framework
Mensagens de validação em inglês	Consistência independente do idioma do servidor
Até ~1 s entre a criação e a primeira chamada ao provedor	Intervalo do worker (configurável)
Depois de falhas seguidas do provedor, solicitações novas ficam `PENDING` por até ~30 s	Circuit breaker aberto: o worker não chama o provedor e ninguém gasta tentativa

9. Registro de defeitos

O `traceId` é o campo mais importante: com ele o desenvolvedor encontra a requisição exata nos logs em segundos.

Exemplo fictício de registro (ilustra o formato; não é um defeito real):

```
Caso:        PRO-03
Resumo:      Pedido com UNAVAILABLE-PRODUCT fica PENDING após 5 tentativas
Ambiente:    commit 1fa88f8 · docker compose local
Passos:      POST {externalId: QA-PRO-03, customerId: CUST-1, productCode: UNAVAILABLE-
PRODUCT};
             GET pelo id após 60 s
Esperado:    FAILED, failureReason "Gave up after 5 attempts..."
Obtido:      PENDING, sem novas tentativas no log
traceId:     4bf92f3577b34da6a3ce929d0e0e4736
Evidência:   print da resposta + trecho do log
Severidade:  Alta · Prioridade: P1
```

Severidade	Quando usar
Alta	Dado errado, perdido ou duplicado; ativação indevida; acesso entre sistemas; `500` em fluxo normal; pedido parado para sempre
Média	Código HTTP, mensagem ou formato diferente do especificado, sem impacto em dados
Baixa	Texto, documentação, cosmético

10. Matriz de rastreabilidade

Resumo por área: requisitos da especificação (`docs/spec/requirements.md`), casos manuais e cobertura automatizada.

Área	Requisitos	Casos	P1	Automatizados	Parcial	Só manual
Autenticação e autorização	R8	9	5	8	0	1
Criação e validação	R1	10	2	6	1	3
Duplicidade e idempotência	R4	7	4	5	0	2
Consulta e isolamento	R2, R8.4	6	2	5	1	0
Listagem, filtros e paginação	R3	9	1	7	1	1
Processamento e integração com o provedor	R5	9	6	8	0	1
Resiliência e operação	R5.7, R9	8	2	2	4	2
Formato de erros e limites	R7, R8.5	9	0	8	0	1
Total		**67**	**22**	**49**	**7**	**11**

Os casos `manual` concentram-se em comportamento de infraestrutura (probes, banco fora, métricas) e em variações de entrada de baixo risco — exatamente onde o teste manual no ambiente integrado agrega mais.